

# HaptiCS: Counter Strike 2 Waffen Feedback

Julius Neun<sup>\*</sup> und Sascha Lang<sup>†</sup>

## 1. Zusammenfassung

Haptisches Feedback sorgt für eine deutlich bessere Immersion bei vielen Videospielen. Allerdings besitzen einige Spiele kein natives haptisches Feedback. Oft liegt es daran, dass keine Möglichkeit besteht dieses Feedback darzustellen. Beispielsweise bei Computerspielen, da üblicher Weise eine Maus und eine Tastatur keine Vibrationsmotoren oder ähnliches besitzen.

Counter Strike 2 ist eines dieser Computerspiele und besitzt kein natives haptisches Feedback. Es wird mit visuellem oder auditivem Feedback gearbeitet. Dieses Feedback kann in stressigen Momenten während des Spiels allerdings für manche Menschen schnell untergehen.

In dieser Arbeit präsentieren wir HaptiCS. Zwei Armbänder die mit Vibrationsmotoren ausgestattet sind und eine haptische Benutzerschnittstelle für Counter Strike 2 darstellen. Das Ziel dieser Benutzerschnittstelle ist eine erhöhte Immersion, sowie auch eine verbesserte Spielerfahrung zu erzeugen.

Die Motoren der Armbänder nutzen Tactile Phantom Sensation sowie verschiedene Vibrationskurven, -intensität und -frequenzen um den Füllstand des Magazins darzustellen und den Nutzer\*innen das Gefühl zu geben tatsächlich die Waffe im Spiel abzufeuern.

## 2. Einleitung

Unser Team hat sich mit der Entwicklung einer haptischen Benutzerschnittstelle für das Videospiel Counter Strike 2 beschäftigt. Die Entwicklung des Prototypen dieser Benutzerschnittstelle stand einen großteil mit eigenem Interesse in Verbindung. Zusätzlich zu unserem eigenen Interesse stellte für uns die Verbesserung der Immersion, sowie auch die verbesserte Wahrnehmung der Geschehnisse innerhalb des Spiels weitere Motivation dar.

Counter Strike 2 nutzt zwar visuelles und auditives Feedback, jedoch nicht haptisches Feedback. Beispielsweise wird durch die Verbindung eines sogenannten „Killfeed“ und dem Ausgrauen des Profilbilds in einer Team Liste als visuelles Feedback angezeigt, ob ein Teammitglied oder eine Gegenspieler\*in im Spiel getötet wurde. Weiterhin wäre ein Beispiel für auditives Feedback, dass nachdem

die Bombe platziert wurde, also eines der beiden Teams deren Ziel sehr nahe sind, durch ein sich beschleunigendes Piepen die Verbleibende Zeit dargestellt wird. Dieses Feedback unterstützt zwar die Nutzer\*innen ungemein, kann allerdings in stressigen Momenten, beispielsweise wenn man der oder die letzte Überlebende im eigenen Team ist, verpasst werden. Haptisches Feedback unterstützt dabei diese Situationen zu vermeiden, da eine Vibration oder ein ähnliches haptisches Signal zusätzliche Aufmerksamkeit erzeugt, und eine Spieler\*in näher and das Spielgeschehen bringt.

Im Laufe dieses Kurses haben wir einen Prototypen einer haptischen Benutzerschnittstelle für Counter Strike 2 namens HaptiCS entwickelt. Dieser Prototyp umfasst einen kleinen Teil der eigentlich entworfenen haptischen Benutzerschnittstelle. Durch zwei mit Vibrationsmotoren ausgestattete Armbänder wird der Rückschlag der im Spiel genutzten Schusswaffe dargestellt. Weiterhin nutzen die Motoren Tactile Phantom Sensation um den Vibrationspunkt näher an die Hand der Nutzer\*in „wandern“ zu lassen. Dieser Effekt soll auf einen sinkenden Munitionswert Aufmerksamkeit erzeugen. Um die Daten zu erhalten wird eine sogenannte Game State Integration (GSI) genutzt, die durch Valve, das Entwicklerstudio hinter Counter Strike 2, bereitgestellt wird. Außerdem haben wir einen Plotter entwickelt, der die Vibration der Motoren visuell darstellt. Angesteuert werden die Motoren über einen, mit einem MOSFET PCB verbundenen, Adafruit ESP32 Feather V2.

In den folgenden Kapiteln werden wir ein Beispielszenario skizzieren und näher auf HaptiCS und dessen Funktionalität, Aufbau und Programmierung eingehen. Außerdem werden wir über unsere Entscheidungen bezüglich HaptiCS, unsere Erkenntnisse, sowie auch unsere Erfahrungswerte diskutieren und auch über Limitierungen unseres Prototyps berichten.

## 3. Beispielszenario

Ana möchte ihr Counter Strike 2 Ranking erhöhen. Ihre Fähigkeiten würden das auch erlauben. Allerdings hat sie große Probleme den Überblick über ihre restliche Munition in ihrem Magazin zu behalten. Sie merkt, dass, wenn sie sich zu sehr auf das Zielen und ihre Position fokussiert, ihr es schwer fällt auf die Munitionsanzeige

---

<sup>\*</sup>Matrikelnummer: 5435386

<sup>†</sup>Matrikelnummer: 5315826

zu schauen. Durch einen Freund wird sie Aufmerksam auf die HaptiCS Ärmbänder und kauft sich diese um sie auszuprobieren. Durch das zusätzliche Feedback wird sie deutlich aufmerksamer auf ihre Munitionsmenge, merkt allerdings auch, dass sich ihr Fokus auf das Zielen und ihre Position nicht verringert.

## 4. HaptiCS

Dieses Kapitel enthält alle relevanten Informationen über HaptiCS, die haptischen Armbänder für Counter Strike 2. Der vollständige Quellcode befindet sich im GitHub-Repository [1].

### 4.1. Systemübersicht

HaptiCS nutzt zwei Vibrationsmotoren, die an zwei frei am Unterarm platzierbaren Armbändern befestigt sind. Gesteuert werden diese Motoren über einen Adafruit ESP32 Feather V2 in Verbindung mit einem MOSFET-PCB. Wir haben die Platzierung am Unterarm bedacht um möglichst genau den Rückschlag der Waffe im Spiel darzustellen. Armband 1 sollte näher an der Hand platziert werden als Armband 2, da zur Darstellung des Magazin Füllstandes die Vibration per Tactile Phantom Sensation von Armband 2 zu Armband 1 wandert.

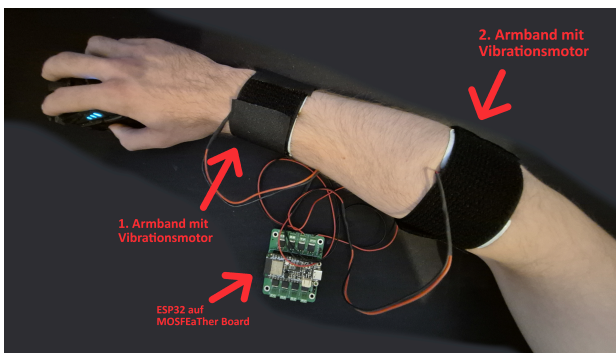


Abbildung 1: Befestigung der HaptiCS Benutzerschnittstelle am Körper

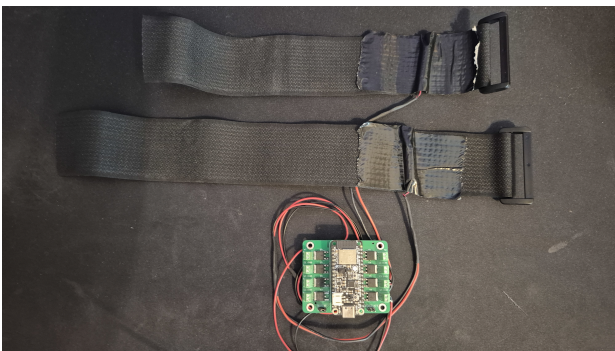


Abbildung 2: Innenseite der HaptiCS Armbänder

Der Microcontroller kommuniziert über eine serielle Schnittstelle mit dem Host-Computer. Auf diesem wird ein Python Skript ausgeführt, das in Verbindung mit einer .cfg Datei, die in den Counter Strike 2 Instal-

lationsordner abgelegt werden muss, die Notwendigen Daten sammelt und an den Microcontroller sendet. Die .cfg Datei aktiviert das Senden der notwendigen Spiel Informationen, die dann durch HTTP-Requests ausgelesen werden können. Die Informationen dieser HTTP-Requests werden nach Erhalt in JSON-Objekten zusammengefasst, in einer eigen entwickelten State Engine, umgesetzt als Python Skript, verarbeitet und danach an den Microcontroller gesendet. Der Microcontroller führt keine Berechnungen durch, er nutzt nur die in den JSON-Objekten enthaltene Daten um die Vibration der Motoren anzupassen.

Die relevanten Daten abgeleitet aus der HTTP-Requests beinhalten:

- `activeWeaponName`, die Bezeichnung der aktuell ausgerüsteten Waffe
- `ammoRatio`, der Füllstand des Magazins der aktuell ausgerüsteten Waffe
- `ammoReduced`, hat sich der Füllstand des Magazins aktuell geändert?
- `alive`, ist die Spieler\*in zurzeit am Leben?

Die JSON-Objekte setzen sich aus Daten aus einer Konfigurations Datei und den erhaltenen Daten aus den HTTP-Requests zusammen. Im Detail sind das:

- `isFiring`, wird die Waffe zur Zeit betätigt? Dieser Zustand wird durch `ammoReduced` aus den HTTP-Requests bestimmt.
- `firingPeriod`, entnommen aus der Konfigurations Datei basierend auf der Aktuell ausgerüsteten Waffe.
- `firingPattern`, entnommen aus der Konfigurations Datei basierend auf der Aktuell ausgerüsteten Waffe.
- `ammunitionPercent`, entnommen aus den erhaltenen Daten der HTTP-Requests.
- `intensityPercent`, entnommen aus der Konfigurations Datei basierend auf der Aktuell ausgerüsteten Waffe.
- `oneShot`, entnommen aus der Konfigurations Datei basierend auf der Aktuell ausgerüsteten Waffe.

Die Inhalte der Konfigurations Datei werden im nächsten Kapitel genauer beleuchtet.

Weiterhin wird auf dem Host-Computer über einen selbst entwickelten Daten Plotter die Vibrationsaktivität der Motoren dargestellt, sowie auch in der Konsole der Inhalt der JSON-Objekte ausgegeben.

Zum Zusammenfassen der Daten in JSON-Objekten nutzt das System `ArduinoJson` (Entwickelt von Benoit Blanchon). Weiterhin verwendet das System die folgenden Python Plug-ins:

- `pyserial` zur Kommunikation über die serielle Schnittstelle

- PyQt5 und die Erweiterung pyqtgraph zur Erzeugung des Plotters
- numpy sowie Flask in dem Verarbeitungsskript

## 4.2. Haptische Stimuli

Die haptischen Signale die HaptiCS nutzt werden durch die Verbindung der im vorherigen Kapitel bereits erwähnten Daten innerhalb der JSON-Dateien erzeugt. Die `firingPeriod`, orientiert an der Schussfrequenz der Waffe, bestimmt die Länge der Vibrationsimpulse. Diese werden erzeugt durch das Verrechnen der `firingPattern`, `ammunitionPercent` und der `intensityPercent`. Diese Werte, ausgenommen `ammunitionPercent`, sind in einer Konfigurationsdatei für jede Waffe festgelegt.

```
WEAPON_DATA = {
    # Pistols
    "weapon_glock": {"hz": 6.67, "firingPattern": 0, "oneShot": True, "intensity": 0.58},
    "weapon_usp_silencer": {"hz": 6.67, "firingPattern": 0, "oneShot": True, "intensity": 0.52},
    "weapon_hkp2000": {"hz": 5.88, "firingPattern": 0, "oneShot": True, "intensity": 0.55},
    "weapon_p250": {"hz": 6.67, "firingPattern": 4, "oneShot": True, "intensity": 0.68},
    "weapon_fiveseven": {"hz": 6.67, "firingPattern": 0, "oneShot": True, "intensity": 0.58},
    "weapon_cz75a": {"hz": 10.91, "firingPattern": 1, "oneShot": False, "intensity": 0.62},
    "weapon_tec9": {"hz": 8.57, "firingPattern": 0, "oneShot": True, "intensity": 0.65},
    "weapon_elite": {"hz": 8.00, "firingPattern": 0, "oneShot": True, "intensity": 0.55},
    "weapon_deagle": {"hz": 4.44, "firingPattern": 4, "oneShot": True, "intensity": 0.90},
    "weapon_revolver": {"hz": 2.00, "firingPattern": 4, "oneShot": True, "intensity": 0.95},

    # SMGs
    "weapon_mac10": {"hz": 13.33, "firingPattern": 2, "oneShot": False, "intensity": 0.59},
    "weapon_mp9": {"hz": 14.20, "firingPattern": 2, "oneShot": False, "intensity": 0.60},
    "weapon_mp7": {"hz": 13.33, "firingPattern": 2, "oneShot": False, "intensity": 0.65},
    "weapon_mp5sd": {"hz": 12.50, "firingPattern": 2, "oneShot": False, "intensity": 0.62},
    "weapon_ump45": {"hz": 11.11, "firingPattern": 2, "oneShot": False, "intensity": 0.75},
    "weapon_p90": {"hz": 14.20, "firingPattern": 2, "oneShot": False, "intensity": 0.68},
    "weapon_bizon": {"hz": 12.50, "firingPattern": 2, "oneShot": False, "intensity": 0.63},

    # Rifles
    "weapon_famas": {"hz": 11.00, "firingPattern": 3, "oneShot": False, "intensity": 0.74},
    "weapon_galliar": {"hz": 10.87, "firingPattern": 3, "oneShot": False, "intensity": 0.76},
    "weapon_ak47": {"hz": 10.00, "firingPattern": 3, "oneShot": False, "intensity": 0.90},
    "weapon_m4a1": {"hz": 11.11, "firingPattern": 3, "oneShot": False, "intensity": 0.82},
    "weapon_m4a1_silencer": {"hz": 10.00, "firingPattern": 5, "oneShot": False, "intensity": 0.75},
    "weapon_aug": {"hz": 10.00, "firingPattern": 3, "oneShot": False, "intensity": 0.84},
    "weapon_sg556": {"hz": 10.87, "firingPattern": 3, "oneShot": False, "intensity": 0.86},

    # Snipers
    "weapon_ssg08": {"hz": 0.80, "firingPattern": 4, "oneShot": True, "intensity": 0.88},
    "weapon_awp": {"hz": 0.67, "firingPattern": 4, "oneShot": True, "intensity": 1.00},
    "weapon_scar20": {"hz": 4.00, "firingPattern": 4, "oneShot": False, "intensity": 0.85},
    "weapon_sg1": {"hz": 4.00, "firingPattern": 3, "oneShot": False, "intensity": 0.80},
}
```

Abbildung 3: Auszug aus der Konfigurations Datei

Die `intensityPercent` ist angepasst an die derzeit ausgerüstete Waffe. Es gibt zur Zeit keinen Weg für Nutzer\*innen, ohne Programmiererfahrung und ohne Verständnis der Systemarchitektur, die Gesamtintensität anzupassen.

Die `firingPattern` ist aus einer Auswahl aus fünf verschiedenen Wertekurven, realisiert als Arrays die Werte zwischen 0 und 255 enthalten, für jede Waffe bestimmt. Die fünf bereits bereitgestellten Arrays umfassen folgende Wertekurven:

1. Konstant
2. Sinus Kurve
3. Sägezahnkurve
4. Fallende Sägezahnkurve
5. Eigen entwickelte HeavyShot Kurve

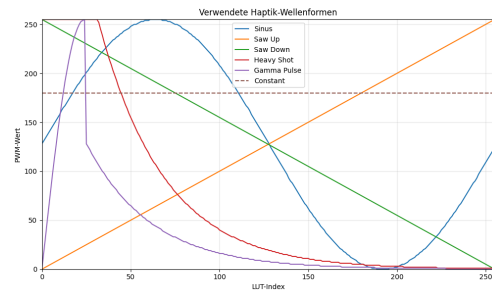


Abbildung 4: Plot der hinterlegten Arrays

Wie bereits erwähnt nutzt HaptiCS die Tactile Phantom Sensation. Diese wird durch das Auslesen des Momentanen Magazin Füllstands der Waffe berechnet. Die Position der Illusion entspricht in etwa dem prozentualen Füllstand. Außerdem wird ein Overdrive verwendet.

## 4.3. Game State Integration

Die Kommunikation zwischen Counter Strike 2 und HaptiCS erfolgt über die von Valve bereitgestellte Game State Integration (GSI). Nach der Installation einer Konfigurationsdatei im Spielverzeichnis sendet Counter Strike 2 Änderungen des Spielzustands als HTTP-Requests an ein Python-Skript auf dem Host-Computer. Dieses verarbeitet die empfangenen Daten, ergänzt sie um waffen-spezifische Konfigurationsparameter und erzeugt daraus kompakte JSON-Objekte für den ESP32.

Durch die Trennung zwischen Host-Computer und Mikrocontroller muss der ESP32 selbst keine aufwendigen Berechnungen durchführen. Die Auswahl des Vibrationsmusters, die Berechnung der Schussperiode sowie die Anpassung der Intensität erfolgen vollständig auf dem Host. Der ESP32 interpretiert lediglich die übertragenen Parameter und erzeugt daraus die entsprechenden PWM-Signale für die Vibrationsmotoren. Dadurch bleibt die Firmware einfach und kann mit einer hohen Aktualisierungsrate arbeiten.

Die Vielfalt der erzeugbaren haptischen Signale entsteht durch das Zusammenspiel dreier Parameter:

- des Vibrationsmusters (Pattern),
- der Schussperiode (Period) und
- der Vibrationsintensität (Intensity).

Obwohl lediglich wenige Grundmuster implementiert wurden, lassen sich durch die Kombination dieser Parameter für jede Waffe deutlich unterschiedliche haptische Charakteristiken erzeugen. Dadurch können sowohl schnelle Maschinenpistolen als auch langsame Präzisionsgewehre mit einem jeweils passenden Vibrationsprofil dargestellt werden.

## 4.4. Kommunikation mit dem ESP32

Die Kommunikation zwischen dem Host-PC und dem ESP32 erfolgt über eine serielle USB-Verbindung. Der

Host-PC verwaltet den vollständigen Spielzustand in Form eines JSON-Objekts. Dieses enthält alle für die Vibrationssteuerung benötigten Parameter, darunter das Vibrationsmuster, die Schussperiode, die Intensität, den aktuellen Magazinfüllstand sowie den OneShot-Modus.

Um die serielle Kommunikation möglichst effizient zu gestalten, werden JSON-Objekte nur dann übertragen, wenn sich der Spielzustand geändert hat. Dadurch wird verhindert, dass identische Zustände fortlaufend an den Mikrocontroller gesendet werden. Eine Ausnahme bildet der OneShot-Modus: Hier wird bei jedem abgegebenen Schuss ein neues Objekt übertragen, auch wenn sich die übrigen Parameter nicht geändert haben. Dadurch erhält der ESP32 für jeden Schuss ein eindeutiges Trigger-Ereignis, während die Anzahl unnötiger Übertragungen dennoch gering bleibt.

Auf dem ESP32 läuft eine separate FreeRTOS-Task, die kontinuierlich die serielle Schnittstelle überwacht. Eingehende Zeichen werden bis zum Zeilenende zwischengespeichert und anschließend als JSON-Objekt interpretiert. Nach erfolgreicher Deserialisierung werden die Daten atomar in die gemeinsam genutzte Zustandsstruktur übernommen. Ein Mutex schützt dabei vor konkurrierenden Zugriffen zwischen der seriellen Empfangsroutine und der Motorsteuerung. Dadurch können neue Spielzustände jederzeit sicher übernommen werden, ohne die Ausgabe der Vibrationssignale zu unterbrechen.

## 5. Diskussion

HaptiCS zeigt, dass sich mit vergleichsweise einfacher Hardware eine leistungsfähige haptische Benutzerschnittstelle für ein Computerspiel realisieren lässt. Besonders hervorzuheben ist die einfache Installation. Neben dem Anlegen der beiden Armbänder müssen lediglich die Game-State-Integration aktiviert und das Python-Programm gestartet werden. Zusätzliche Änderungen am Spiel oder Eingriffe in dessen Programmcode sind nicht erforderlich.

Neben der Steigerung der Immersion bietet HaptiCS einen weiteren praktischen Nutzen. Durch die Darstellung des Magazinfüllstands mittels Tactile Phantom Sensation erhält die Spielerin oder der Spieler kontinuierlich Informationen über die verbleibende Munition, ohne den Blick von der Spielsituation abwenden zu müssen. Dadurch ergänzt das System das bereits vorhandene visuelle und akustische Feedback um einen zusätzlichen Informationskanal.

Da ausschließlich die offiziell von Valve bereitgestellte Game State Integration verwendet wird und keine Veränderungen am Spiel vorgenommen werden, kann das System grundsätzlich auch in kompetitiven Matches oder Turnieren eingesetzt werden. Es handelt sich um eine ex-

terne Benutzerschnittstelle, die lediglich bereits verfügbare Spielinformationen in haptisches Feedback umsetzt.

## 6. Limitierungen

Aufgrund des begrenzten zeitlichen Rahmens konnten einige ursprünglich geplante Funktionen nicht mehr umgesetzt werden.

Eine wichtige Erweiterung wäre eine direkt am Gerät einstellbare Gesamtintensität der Vibrationen gewesen. Derzeit kann diese lediglich durch große Änderungen im Python-Code angepasst werden. Für eine alltagstaugliche Nutzung wäre stattdessen ein Potentiometer oder eine vergleichbare Hardwarelösung sinnvoll, mit der die Intensität jederzeit individuell eingestellt werden könnte.

Darüber hinaus unterstützt die verwendete Game State Integration zahlreiche weitere Spielereignisse, die bislang nicht genutzt werden. Beispielsweise könnten Treffer, Kills, Granatenwürfe, das Platzieren oder Entschärfen der Bombe oder Schadensereignisse ebenfalls durch unterschiedliche haptische Signale dargestellt werden. Dadurch ließe sich der Informationsgehalt der Benutzerschnittstelle erheblich erweitern.

Eine weitere Einschränkung besteht darin, dass der aktuelle Prototyp ausschließlich den Rückstoß und den Magazinfüllstand der Waffe abbildet. Andere Aspekte des Spielgeschehens werden bislang nicht berücksichtigt.

## 7. Fazit

Mit HaptiCS wurde erfolgreich ein funktionsfähiger Prototyp einer haptischen Benutzerschnittstelle für Counter Strike 2 entwickelt. Durch die Kombination aus unterschiedlichen Vibrationsmustern, variabler Schussperiode und an die jeweilige Waffe angepasster Intensität entstehen realistische und gut unterscheidbare haptische Signale. Gleichzeitig vermittelt die Nutzung der Tactile Phantom Sensation kontinuierlich Informationen über den aktuellen Magazinfüllstand und erweitert damit das vorhandene visuelle und akustische Feedback des Spiels.

Der entwickelte Prototyp zeigt, dass sich haptisches Feedback mit geringem Hardwareaufwand in bestehenden Computerspielen integrieren lässt und sowohl die Immersion als auch den Informationsgewinn erhöhen kann. Die modulare Architektur aus Game State Integration, Python-Anwendung und ESP32 ermöglicht zudem eine einfache Erweiterung um weitere Spielfunktionen und haptische Ereignisse. Damit bildet HaptiCS eine geeignete Grundlage für zukünftige Entwicklungen im Bereich haptischer Benutzerschnittstellen für Computerspiele.



## **Bibliografie**

- [1] J. Neun und S. Lang, „CS2-Haptic-intigration“. [Online]. Verfügbar unter: <https://github.com/Juli-9/CS2-Haptic-intigration>